Decentralized Verification of Multiparty Computations Using Elliptic-Curve Aggregation

Kenneth Hurley

https://super-genius.github.io

Independent technical manuscript | Version 1.0 | November 1, 2026

Abstract

Decentralized systems must determine whether a collection of participants has produced a valid result without necessarily disclosing each participant's input. This paper describes a proposed verification architecture built around a requestor, elliptic-curve aggregates, and a circuit-oriented transaction validator. It gives the relations checked by a supplied C++ prototype, distinguishes group arithmetic from a complete zero-knowledge proof, and identifies the conditions required for private, independently verifiable execution. The design supports a broader goal of checking distributed transaction or compute results at network boundaries. The accompanying source is a prototype; this paper does not report a production deployment, cryptographic security proof, or performance benchmark.

Keywords: multiparty computation; elliptic-curve commitments; transaction verification; decentralized systems

1 Introduction

Distributed computation introduces a verification problem: parties can submit intermediate values or results, but peers need a sound way to accept a result despite node failure, disagreement, or malicious input. A direct recomputation may be expensive and can reveal private values. Cryptographic commitments and proofs offer ways to separate the values being checked from the data disclosed to a verifier.

The architecture examined here arose from a proposed decentralized multiparty-computation (MPC) verifier and two C++ modules: `Requestor` and the MPC verifier circuit. Its narrow implementation concerns transaction aggregates and balance checks. A larger design contemplates distributed proof generation, independent verification, and optional consensus acceptance. We distinguish these layers because arithmetic consistency alone does not establish general MPC correctness or secrecy.

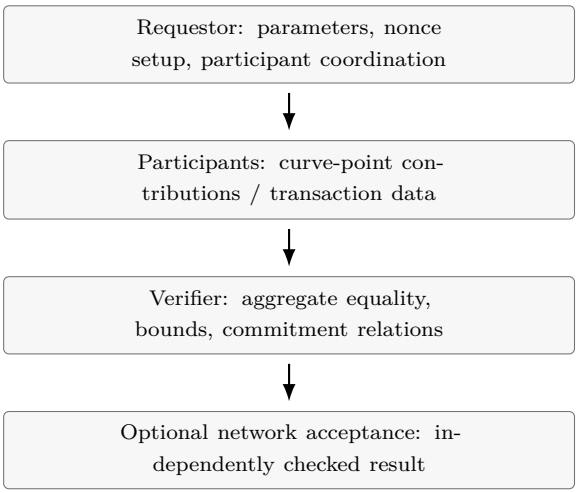


Figure 1: Logical layers of the proposed verification architecture. The last layer is an integration concept, not demonstrated by the supplied circuit.

2 System Model

Let $P_1, \ldots, P_n$ be participants and let V be a verifier. A requestor initializes parameters, distributes or derives nonce material, and collects an aggregate. A transaction consists of an existing balance b, an amount a, and a proposed new balance $b' = b - a$. The public verification inputs in the prototype are curve-point commitments and an aggregate point; the circuit marks some scalar values as private inputs.

The prototype uses the Pallas elliptic-curve type from the Nil Crypto3 library. Let G denote the chosen curve generator and $\mathbb{F}_q$ the relevant scalar field. Curve points form an additive group. An expression such as xG denotes scalar multiplication, not a native 256-bit integer accumulator.

3 Verification Relations

The supplied `MPCValidateTransaction` routine returns the conjunction of five checks:

$$A_{\text{out}} = A_{\text{expected}}, \tag{1}$$
$$b \geq a, \tag{2}$$
$$C_b = bG, \tag{3}$$
$$C_a = aG, \tag{4}$$
$$C_{b'} = (b-a)G. \tag{5}$$

Here A_{out} is the supplied final aggregate point and A_{expected} is an expected aggregate point. The inequalities and commitment equalities are explicit in the source. They give a compact transaction consistency test when the caller's inputs and group operations have the intended meaning.

These relations are *not* by themselves a completed zero-knowledge proof. A sound proof implementation must define a proof relation, trusted or transparent setup assumptions, a prover, a verifier, and an exact binding of each public statement to private witnesses. A `[[circuit]]` annotation indicates design intent but does not prove that those steps are implemented in the files provided.

3.1 Nonce aggregation

The requestor constructs two point sequences using a base nonce ν, a node-dependent value r_i, a node index i, and a block index j. In simplified notation, its pair construction has the form

$$N^{-}_{i,j} = (\nu - r_i + i + j)G, \tag{6}$$
$$N^{+}_{i,j} = (\nu + r_i + i + j)G. \tag{7}$$

Adding one matched pair cancels r_i:

$$N^{-}_{i,j} + N^{+}_{i,j} = 2(\nu + i + j)G. \tag{8}$$

This cancellation is an algebraic property of the construction; it does not hide the inputs against all attacks. It also means the paired total is not simply a sum of the r_i values.

The current requestor stores values across node and block loops, but its `getFinalAggregate()` iterates over only the first n stored pairs. A separate method computes $2\nu nmG$, where m is the number of blocks per node. These expressions need reconciliation before the aggregate can serve as a reliable public statement.

4 Privacy and Correctness

4.1 Commitment design

The sample uses $C_v = vG$. For a small amount or balance, a party can enumerate candidate v values and compare points, so this representation alone does not provide strong hiding. A standard blinded alternative is a Pedersen-style commitment,

$$C_v = vG + rH, \tag{9}$$

with independent generators G, H whose discrete-log relation is unknown and a fresh random blinding scalar r [1]. Such a change would require corresponding verification constraints and a correct mechanism for opening or proving relations among commitments.

4.2 Range and overflow claims

Embedding a 32-bit or 64-bit value into a 256-bit group does not automatically prevent overflow in an integer calculation. Group arithmetic is modulo the group order. To show that a balance remains nonnegative in ordinary integer arithmetic, a proof must constrain the range of a, b, and $b - a$, including any carry or wraparound conditions. Efficient range proofs have a substantial prior literature [2].

4.3 Adversarial nodes

A verifier should reject stale transcripts, reused nonces, malformed or off-curve points, inconsistent participant identities, unauthorized changes to the accepted statement, and aggregates assembled from incomplete participant sets. Proofs should bind to a request identifier, participant set, network identifier, and fresh challenge. Cryptographic nonce material must come from a cryptographically secure source. The prototype uses `std::mt19937`, which is appropriate for simulation but not secret cryptographic randomness.

5 Distributed Protocol

A complete protocol can be organized into four stages. First, the requestor establishes a signed request context and a precisely defined algebraic statement. Second, participants submit authenticated contributions and, where needed, encrypted or committed private values. Third, a proof system produces evidence that valid contributions imply a result satisfying the stated arithmetic and policy constraints. Fourth, an independent verifier checks the proof and authorizes the network to accept or reject the result.

The precise proof system remains an engineering choice. Circuit-based proofs could use existing succinct-proof systems, while simpler arithmetic relations may admit specialized sigma protocols or range proofs. Proof construction should reuse audited primitives rather than rely on new cryptographic assumptions.

6 Evaluation Plan and Implementation Status

The available test file proposes five cases: a valid transfer, insufficient balance, invalid commitments, a zero-valued transfer, and a transfer equal to the full balance. Tests are useful for exercising the Boolean predicate, but no results, timings, proof sizes, or network measurements were supplied. The following evaluation should precede any empirical claims.

Measure	Required observation
Correctness	Pass/fail across ordinary and adversarial inputs
Scalability	Cost as participants and blocks grow
Proof cost	Prover time, verifier time, proof bytes
Fault behavior	Dropped nodes, reordered messages, replay
Privacy	Witness leakage and linkability analysis

Table 1: Measurements needed to turn the design into a quantified evaluation.

Before those measurements, the prototype needs a valid library-provided curve generator, consistent aggregation indices and formulas, secure randomness, and a concrete proof-generation and proof-verification path. All conclusions in this manuscript are therefore architectural and algebraic rather than empirical.

7 Distributed Compute Uses

In a decentralized compute system, verifiable arithmetic could support job settlement, quota tracking, resource accounting, or signed acceptance records. It does not establish that an arbitrary AI answer is factually correct or that arbitrary unproved computation was executed faithfully. Each application must define the property being proved and how the public result binds to the original job.

8 Conclusion

The requestor and circuit modules illustrate an approach for checking aggregate and transaction consistency with elliptic-curve operations. The next step is a precise security relation and a working proof system, followed by testing across adversarial network conditions. A limited, accurately stated verifier is more useful than an unverified claim of general MPC correctness.

References

[1] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Advances in Cryptology—CRYPTO '91*, 1992.

[2] B. Bünz et al., "Bulletproofs: Short proofs for confidential transactions and more," in *IEEE Symposium on Security and Privacy*, 2018.

[3] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, 1979.

[4] Electric Coin Company, "The Pasta curves for Halo 2 and beyond," technical documentation, 2020.

[5] K. Hurley, *MPCVerifierCircuit.cpp*, *Requestor.cpp*, and accompanying test and header files, author-provided prototype, 2024–2025.

Prepared for publication at https://super-genius.github.io. This work is a technical exposition, not a claim of independent certification or completed performance evaluation.